

Réflexion initiale technologique - Vite & Gourmand

1. Choix du framework backend : Symfony (PHP) vs Laravel

Contexte de la décision

En amont du développement, alors que les maquettes et diagrammes (MCD, cas d'utilisation, séquence) étaient finalisés mais que je n'avais pas commencé à coder, une remise en question de Symfony au profit de Laravel a été envisagée, dans l'objectif de gagner du temps sur un délai restreint.

Arguments en faveur d'un changement vers Laravel, après prise d'information sur les forums et sites officiels :

- Courbe d'apprentissage réputée plus douce pour un profil débutant (qui me correspond)
- ORM Eloquent présenté comme plus intuitif que Doctrine
- Conventions par défaut (Artisan) permettant d'aller vite sur des projets simples

Arguments en faveur du maintien de Symfony (solution que j'ai retenue) :

- Aucun code n'était encore écrit dans l'un ou l'autre framework : le changement n'aurait pas eu d'incidence sur un éventuel développement réalisé, mais aurait nécessité un réapprentissage complet sous pression de délai. Cela aurait représenté une prise de risque plutôt qu'un bonus.
- J'ai réalisé que le gain de vitesse attribué à Laravel provient surtout d'une familiarité avec ses conventions, pas d'une simplicité avérée pour un apprenant.
- Les diagrammes de modélisation sont indépendants du framework : ils restaient valables quel que soit le choix technique.

Décision finale

Maintien de Symfony 7 / Doctrine ORM, validé par un test pratique sur un mini-projet de blog avant d'engager le développement du projet principal.

2. Choix du format de modélisation

J'ai choisi d'effectuer la réalisation du MCD avec l'outil Looping, qui permet une vérification visuelle claire des entités, relations et cardinalités avant le passage au code.

Il s'agit, par ailleurs, du modèle avec lequel je suis la plus à l'aise, bien que la lecture des cardinalités soit différente des autres modèles (cardinalité côté de l'entité décrite).

J'ai réalisé les diagrammes de séquence et de cas d'utilisation avec draw.io, pour sa simplicité d'utilisation.

3. Mini projets de sécurisation préalables :

Portfolio personnel (PHP/PDO) - requêtes préparées, résolution de bugs classiques.

Mini-blog (Symfony 7) - entités Doctrine, formulaires (création d'articles, connexion), authentification, résolution de problématiques de configuration, directement réinvesties sur le

projet principal.

4. Architecture avec deux bases de données (MySQL + MongoDB)

Cette division était imposée.

Plutôt que de recalculer les statistiques à chaque fois en interrogeant plusieurs tables MySQL liées entre elles, j'enregistre directement, dans MongoDB, une copie simplifiée de chaque commande (juste le menu, la date et le montant) au moment où elle est validée. Cette copie est pensée uniquement pour calculer rapidement les statistiques. MySQL gère ainsi les vraies données de l'activité (commandes, utilisateurs, menus), qui doivent être fiables et complètes, tandis que MongoDB gère les statistiques, optimisées pour être lues rapidement.